Escalabilidad del Quantum Analysis Framework

De un millón a cincuenta millones de variables: estructura, memoria, conectividad y regímenes computacionales

Juan Manuel Goenaga

Manuscrito técnico de validación y exploración computacional

Nota de alcance. Este estudio separa deliberadamente dos niveles: (i) leyes exactas de escalamiento de representaciones densas, bandeadas y MPS; y (ii) una prueba ejecutada en Python de un compilador contextual *streaming* con complejidad local acotada. La ejecución a $K = 50$ millones **no** constituye una ejecución del QAF completo MPS/TDVP ni una prueba de ventaja operacional. Es una prueba de una propiedad necesaria para escalar: procesar variables en tiempo y memoria aproximadamente lineales sin materializar estructuras K^2.

19 de septiembre de 2026

Resumen

El objetivo de este manuscrito es estudiar, con una combinación de análisis matemático y pruebas computacionales reproducibles, qué significa realmente escalar el Quantum Analysis Framework (QAF) desde cientos de variables hasta $K = 1$, 5, 10, 25 y 50 millones de variables. La pregunta central no es si un computador puede "contar" cincuenta millones de variables, sino qué estructura debe preservar entre ellas. Una matriz QUBO densa crece como $\mathcal{O}(K^2)$; un QUBO bandeado de ancho b crece como $\mathcal{O}(Kb)$; y una representación MPS de dimensión de enlace χ requiere memoria $\mathcal{O}(Kd\chi^2)$ y trabajo por barrido del orden de $\mathcal{O}(Kd^3\chi^3)$. Para $K = 50$ millones, una matriz QUBO densa Float64 requeriría aproximadamente 20 PB, mientras que una representación CSR bandeada con $b = 8$ requeriría aproximadamente 14 GB. Un MPS complex64 requeriría aproximadamente 51.2 GB para $\chi = 8$, 204.8 GB para $\chi = 16$, 819.2 GB para $\chi = 32$ y 3.28 TB para $\chi = 64$. En paralelo, se ejecutó un proxy de compilación contextual por bloques, con conectividad local fija, que procesó 50 millones de variables en 1.07 s de media, con un throughput medio global de 45.5 millones de variables por segundo y un incremento máximo observado de memoria residente de aproximadamente 47.9 MB. El ajuste tiempo–K presentó $R^2 = 0.998$, consistente con comportamiento casi lineal en esta prueba. La principal conclusión es que K **no es la variable de escalabilidad dominante por sí sola**: la viabilidad depende del crecimiento de la conectividad efectiva b y, para MPS/TDVP, de la dimensión de enlace χ. El resultado transforma la pregunta de investigación: validar QAF a escala masiva requiere demostrar que el sistema puede mantener b y/o χ acotados mientras conserva información decisional suficiente. Esa propiedad, y no la mera capacidad de enumerar millones de variables, debe convertirse en el siguiente objeto experimental.

Palabras clave: Quantum Analysis Framework, QAF, escalabilidad, MPS, QTT, tensor train, QUBO, grandes sistemas, dimensión de enlace, matrices dispersas, benchmarking.

Índice

1. La pregunta correcta sobre los grandes números **2**

2. Estado del arte: cómo se escala hoy **2**
2.1. Optimización binaria y matrices QUBO 2
2.2. Solvers híbridos BQM 3
2.3. Tensor Train y Matrix Product States 3

3. Hipótesis de escalabilidad del QAF **3**

4. Diseño del experimento **4**
4.1. Escalas estudiadas 4
4.2. Dos pruebas diferentes 4
4.3. Lo que no prueba este experimento 4

5. La explosión combinatoria visible **5**

6. Resultados de memoria: el cambio de régimen **6**
6.1. QUBO denso 6
6.2. Bandeado versus MPS 6

7. El comportamiento de χ: la variable oculta **7**

8. El comportamiento de b: la geometría importa **8**

9. Resultados ejecutados: proxy streaming hasta 50 millones **9**

10. Trabajo TDVP: lineal en K no significa barato **10**

11.Sabidurías halladas **11**
11.1. 1. El gran número relevante no es K, sino K^2 cuando todo interactúa con todo 11
11.2. 2. La estructura vale más que la fuerza bruta 11
11.3. 3. χ es una variable de fase computacional 11
11.4. 4. RCM y localidad deben convertirse en métricas, no en afirmaciones 11
11.5. 5. 50 millones exigen arquitectura jerárquica 12
11.6. 6. La prueba streaming muestra una posibilidad, no una solución completa 12

12.Un programa experimental para validar $K = 50$ millones de forma rigurosa **12**

13.Implicaciones para una propuesta de investigación avanzada **12**

14.Limitaciones **13**

15.Conclusiones **13**

A. Código Python completo de la prueba de escalabilidad **15**

1 La pregunta correcta sobre los grandes números

Cuando K pasa de 500 a 50 millones, la intuición humana tiende a considerar solamente el número de variables. Computacionalmente, ese no es el punto decisivo. Dos sistemas con el mismo K pueden pertenecer a universos de complejidad completamente distintos: uno puede tener interacciones locales y estructura de bajo rango; el otro puede contener interacciones entre casi todos los pares. En el primer caso, la representación puede crecer de forma casi lineal. En el segundo, la descripción misma del problema puede volverse prohibitiva antes de ejecutar cualquier optimizador.

La tesis de este manuscrito puede expresarse en una frase:

> **El problema de 50 millones de variables no es 50 millones; es cuántas relaciones entre esas variables deben mantenerse explícitamente y cuánto rango efectivo necesita la representación para conservarlas.**

Esta forma de mirar el problema sigue una tradición de ciencia computacional: estudiar reglas locales, identificar los parámetros que cambian el régimen global y distinguir el tamaño nominal de un sistema de la complejidad de su estructura.

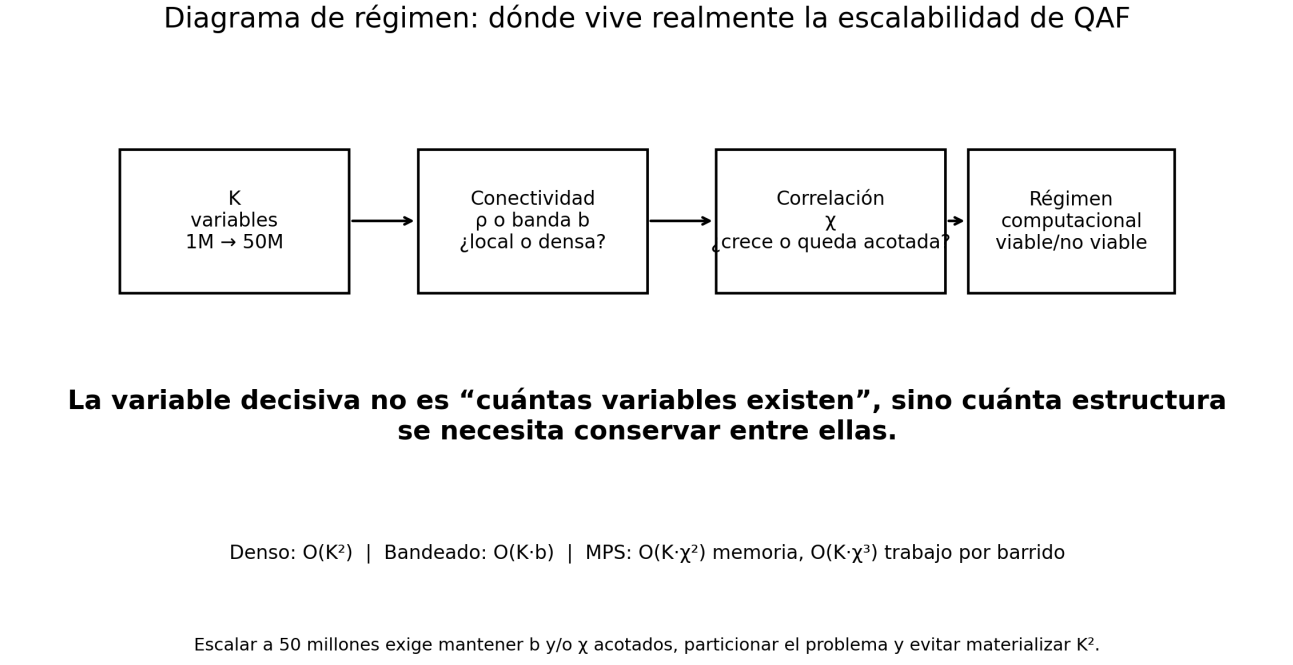


Figura 1: El régimen computacional está gobernado conjuntamente por K, la conectividad y la complejidad de correlación.

2 Estado del arte: cómo se escala hoy

2.1 Optimización binaria y matrices QUBO

Una formulación QUBO representa una función objetivo cuadrática

$$E(x) = x^T Q x, \qquad x \in \{0,1\}^K. \tag{1}$$

Si Q es densa, contiene K^2 posiciones. La complejidad de almacenamiento es entonces $\mathcal{O}(K^2)$. Para $K = 50$ millones, el mero almacenamiento de una matriz Float64 completa exige aproximadamente 20 petabytes. El problema aparece antes del algoritmo de optimización: no es práctico construir el objeto matemático de manera explícita.

Los optimizadores modernos no requieren siempre matrices densas. Gurobi, por ejemplo, permite construir objetivos cuadráticos mediante representaciones dispersas en coordenadas y mantiene métricas explícitas del número de no nulos en modelos grandes [8]. La consecuencia es fundamental: el tamaño K no determina por sí mismo el costo; lo hace también el número de coeficientes no nulos.

2.2 Solvers híbridos BQM

La documentación vigente de D-Wave para su solver híbrido BQM informa un máximo de un millón de variables para una instancia BQM individual [7]. Esto es un referente útil: $K = 1$ millón se encuentra dentro del límite nominal de ese servicio, pero $K = 5$–50 millones excede el tamaño de una instancia única. Por tanto, cualquier propuesta que pretenda operar a esas escalas deberá usar partición, jerarquías, descomposición, ventanas activas o compilación distribuida; no basta con producir un BQM gigante y enviarlo directamente al servicio actual.

2.3 Tensor Train y Matrix Product States

El formato Tensor Train de Oseledets fue propuesto precisamente para evitar la maldición de la dimensionalidad cuando una estructura de bajo rango está disponible [3]. Matrix Product States (MPS) proporciona una representación equivalente en numerosos contextos de sistemas de muchos cuerpos y su utilidad depende críticamente de la dimensión de enlace χ [4]. En una cadena de K sitios y dimensión física d, una cota de almacenamiento típica es

$$M_{\mathrm{MPS}} \propto K d \chi^2. \tag{2}$$

Para evoluciones TDVP, el costo dominante por barrido puede escalar como

$$W_{\mathrm{TDVP}} \propto K d^3 \chi^3, \tag{3}$$

según la estructura del algoritmo y de los operadores [5].

El punto crucial es que estas expresiones son lineales en K *si* χ *permanece acotado*. Si la dinámica obliga a aumentar χ, el sistema puede cambiar de régimen rápidamente. Por ello, la hipótesis científica de escalabilidad no debe formularse como "MPS es lineal", sino como: **¿puede QAF conservar el contenido decisional con un χ suficientemente pequeño y estable?**

3 Hipótesis de escalabilidad del QAF

El manuscrito técnico de QAF propone compresión MPS/QTT, reordenamiento topológico RCM y compilación local mediante matrices de densidad reducida para evitar la expansión del espacio global [1]. A partir de esa arquitectura, formulamos cuatro hipótesis separables:

H1. Representación: si la interacción efectiva es local o bandeable, la memoria del problema puede pasar de $\mathcal{O}(K^2)$ a $\mathcal{O}(Kb)$.

H2. Compresión tensorial: si χ permanece acotado, la memoria MPS permanece aproximadamente lineal en K.

H3. Procesamiento: si las actualizaciones pueden realizarse por ventanas o bloques con conectividad local fija, una etapa de compilación puede mantener throughput aproximadamente constante al aumentar K.

H4. Falsación: si b o χ crecen rápidamente con K o con la incertidumbre contextual, la supuesta escalabilidad se degrada; en ese régimen, el QAF masivo requerirá partición o una arquitectura jerárquica.

Estas hipótesis son más fuertes científicamente que afirmar que "QAF maneja 50 millones de variables", porque especifican *bajo qué condiciones* la escala es posible y qué observación refutaría la hipótesis.

4 Diseño del experimento

4.1 Escalas estudiadas

Se estudiaron cinco tamaños nominales:

$$K \in \{10^6,\ 5\times10^6,\ 10^7,\ 2.5\times10^7,\ 5\times10^7\}. \tag{4}$$

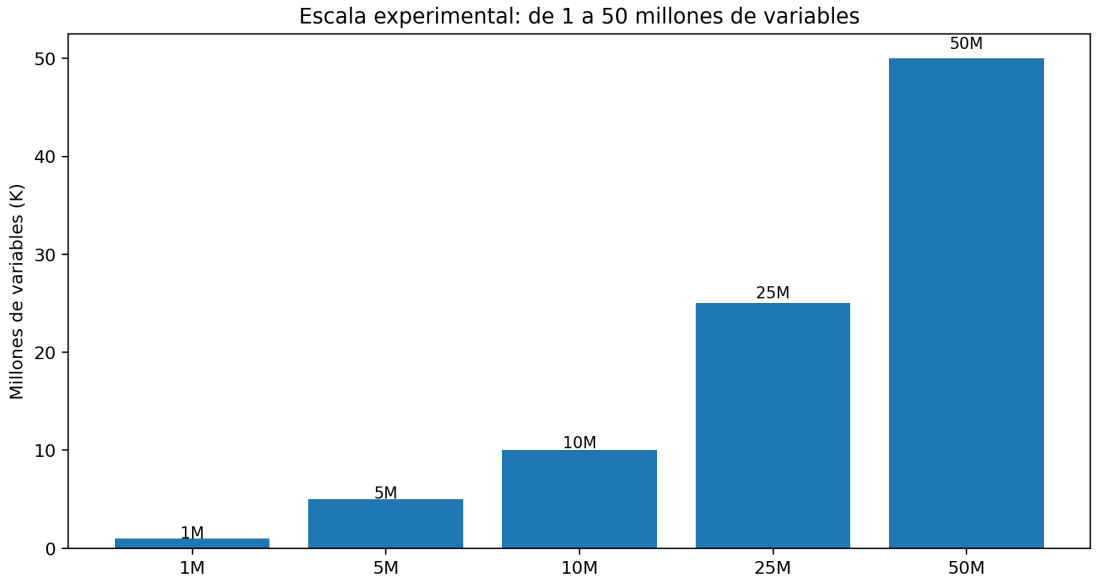


Figura 2: Escalas incluidas en el análisis.

4.2 Dos pruebas diferentes

El estudio tiene dos componentes que no deben confundirse.

Prueba A - Escalamiento analítico exacto. Se calculan número de pares, memoria de QUBO denso, memoria de un QUBO bandeado almacenado en CSR y memoria MPS para varios valores de χ. Estas cifras se derivan de las fórmulas de representación; no son extrapolaciones de solver.

Prueba B - Proxy ejecutado de compilación streaming. Se implementa en Python una pasada por bloques sobre K variables. Para cada variable se generan determinísticamente una confianza, una magnitud de impacto y un ajuste contextual; se calcula un score local y se acumulan interacciones con cuatro offsets fijos. La estructura no construye una matriz $K \times K$ y mantiene un tamaño de bloque fijo. Su complejidad es $\mathcal{O}(Kb)$ con b constante. Esta prueba mide si una operación *local y estructurada* puede conservar comportamiento aproximadamente lineal hasta 50 millones de variables.

4.3 Lo que no prueba este experimento

No se ejecutó TDVP sobre 50 millones de sitios; no se calcularon 1-RDM/2-RDM completas a esa escala; no se resolvió un QUBO de 50 millones de variables; y no se demostró superioridad operacional. El experimento es deliberadamente más estrecho: identifica los regímenes de representación y mide un mecanismo streaming que sería necesario en una arquitectura extrema.

5 La explosión combinatoria visible

El número de pares posibles es

$$P(K) = \frac{K(K-1)}{2}. \tag{5}$$

Para $K = 50$ millones, esto produce aproximadamente 1.25×10^{15} pares. Incluso si un sistema pudiera inspeccionar mil millones de pares por segundo de manera sostenida, recorrer todos los pares una sola vez tomaría aproximadamente 14.47 días. Esta cifra convierte una abstracción matemática en una intuición operativa: la conectividad densa es incompatible con tiempo real a gran escala.

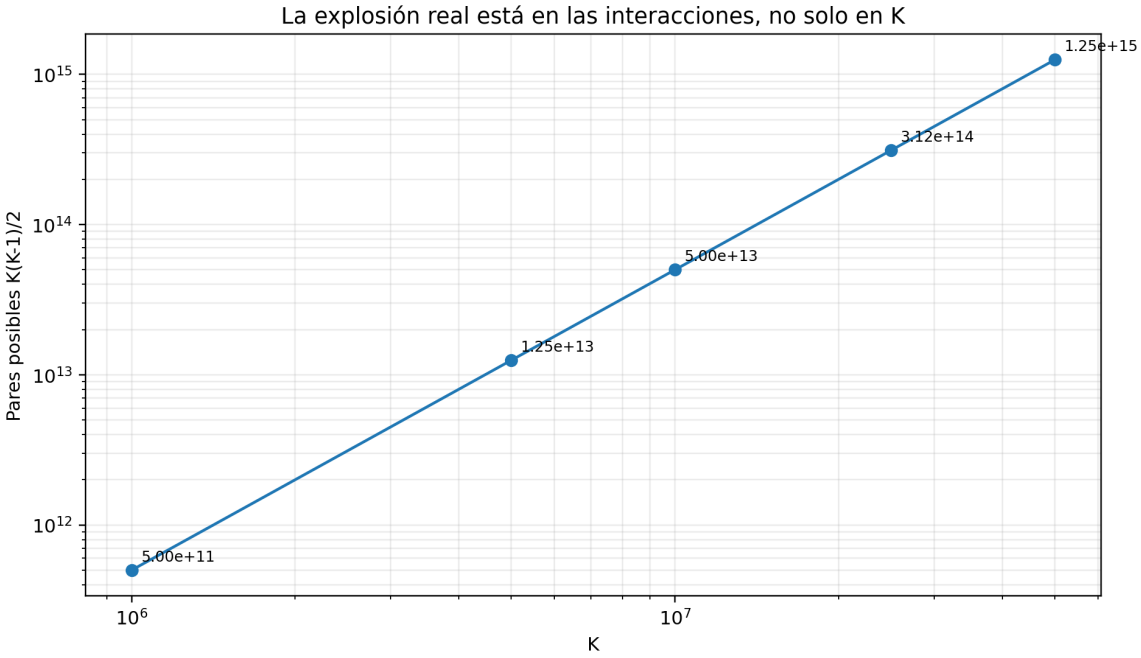


Figura 3: Número de pares posibles al crecer K.

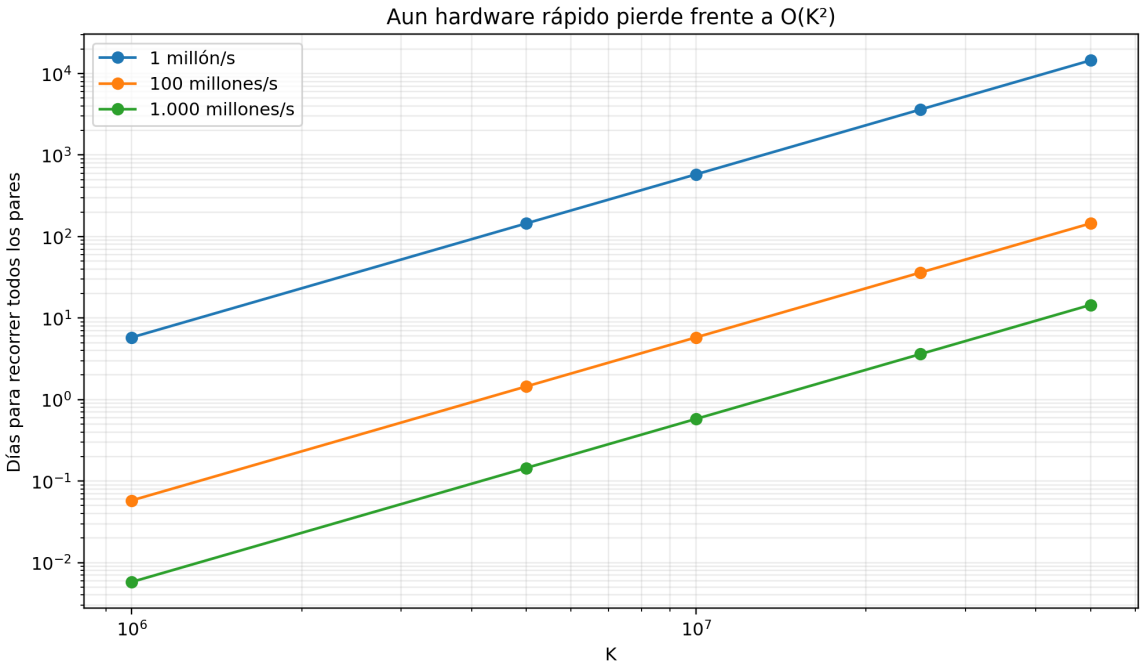


Figura 4: Tiempo teórico para recorrer todos los pares bajo tres tasas de procesamiento.

6 Resultados de memoria: el cambio de régimen

6.1 QUBO denso

La memoria Float64 requerida por un QUBO denso es

$$M_{\text{dense}} = 8K^2 \text{ bytes.} \tag{6}$$

La Tabla 1 muestra la magnitud de la transición.

Cuadro 1: Memoria de representaciones seleccionadas.

K	QUBO denso	CSR $b = 8$	MPS $\chi = 8$	MPS $\chi = 16$	MPS $\chi = 64$
1 M	8 TB	0.28 GB	1.02 GB	4.10 GB	65.54 GB
5 M	200 TB	1.40 GB	5.12 GB	20.48 GB	327.68 GB
10 M	800 TB	2.80 GB	10.24 GB	40.96 GB	655.36 GB
25 M	5 PB	7.00 GB	25.60 GB	102.40 GB	1.64 TB
50 M	20 PB	14.00 GB	51.20 GB	204.80 GB	3.28 TB

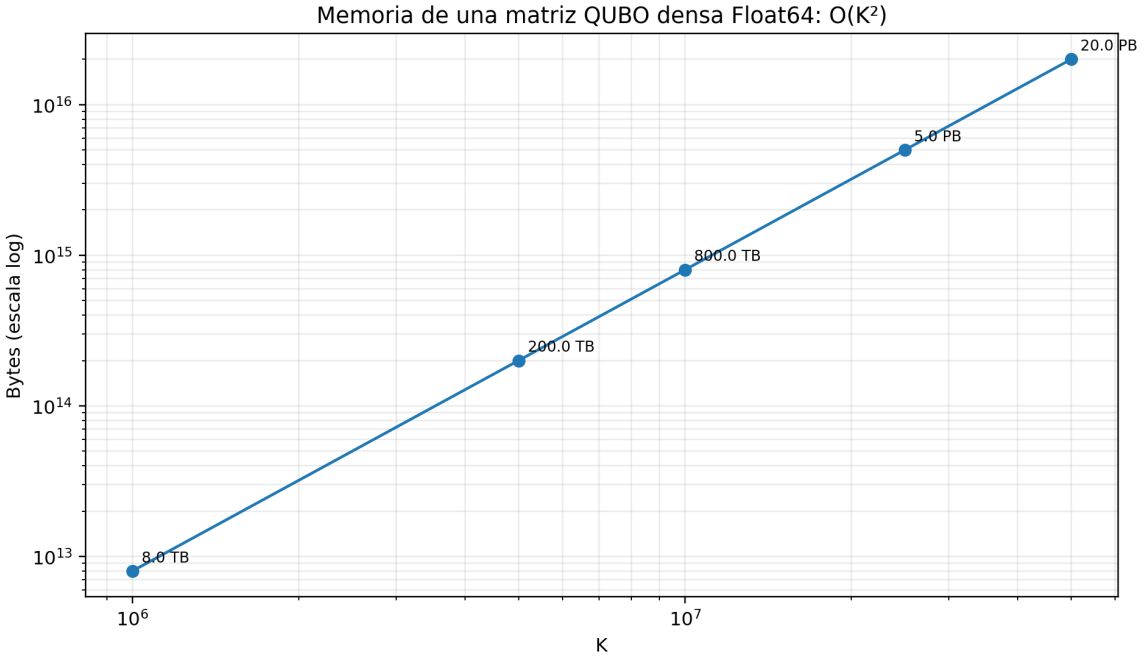


Figura 5: Una representación densa cambia rápidamente de terabytes a petabytes.

6.2 Bandeado versus MPS

La comparación entre representaciones muestra que la pregunta "¿cuántas variables?" es insuficiente. Un QUBO bandeado con $b = 8$ puede representar 50 millones de variables con un orden de 14 GB bajo el modelo CSR conservador adoptado, mientras un MPS con $\chi = 16$ requeriría aproximadamente 205 GB. Ambas son enormemente menores que 20 PB, pero obedecen a supuestos diferentes: el QUBO bandeado presupone localización de coeficientes; MPS presupone compresibilidad del estado mediante bajo rango tensorial.

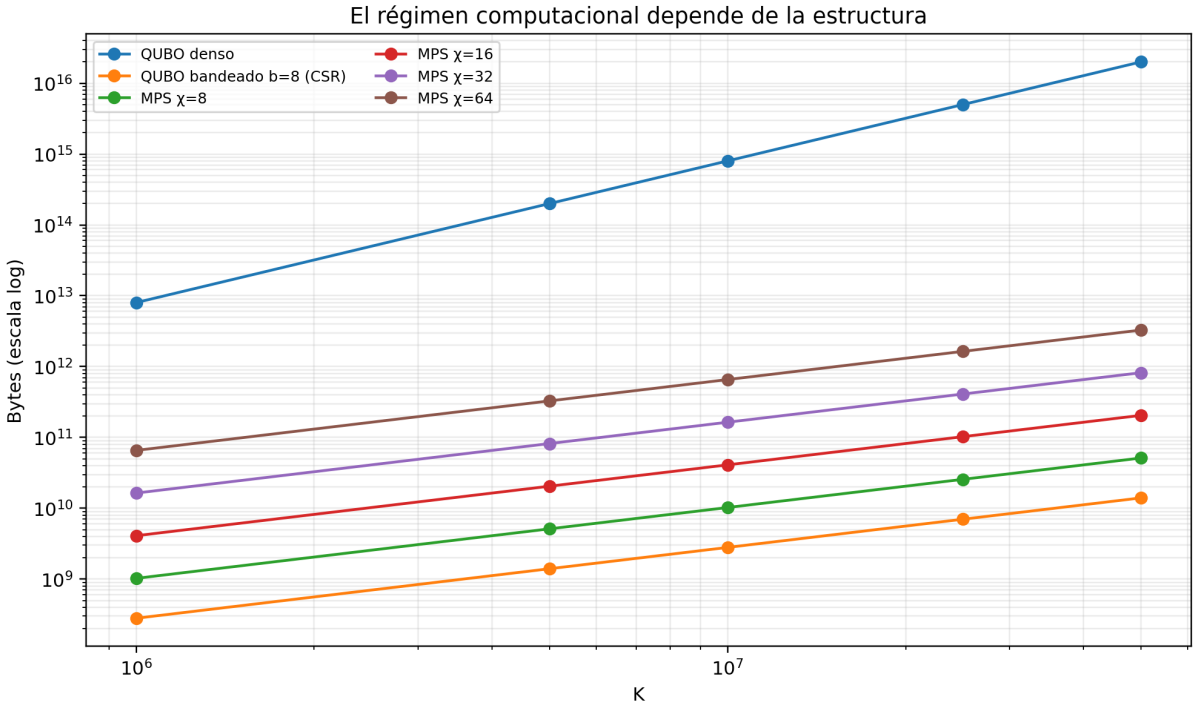


Figura 6: Comparación logarítmica de memoria entre diferentes representaciones.

7 El comportamiento de χ: la variable oculta

La dimensión de enlace χ es una de las variables más importantes del experimento. Mientras K entra linealmente en la memoria MPS, χ entra al cuadrado. Por ello, duplicar K duplica aproximadamente la memoria; duplicar χ la multiplica aproximadamente por cuatro.

En $K = 50$ millones:

- $\chi = 8$: ≈ 51.2 GB;
- $\chi = 16$: ≈ 204.8 GB;
- $\chi = 32$: ≈ 819.2 GB;
- $\chi = 64$: ≈ 3.28 TB.

Esto produce una sabiduría central: **a grandes K, controlar el crecimiento de χ puede ser más importante que reducir K modestamente.**

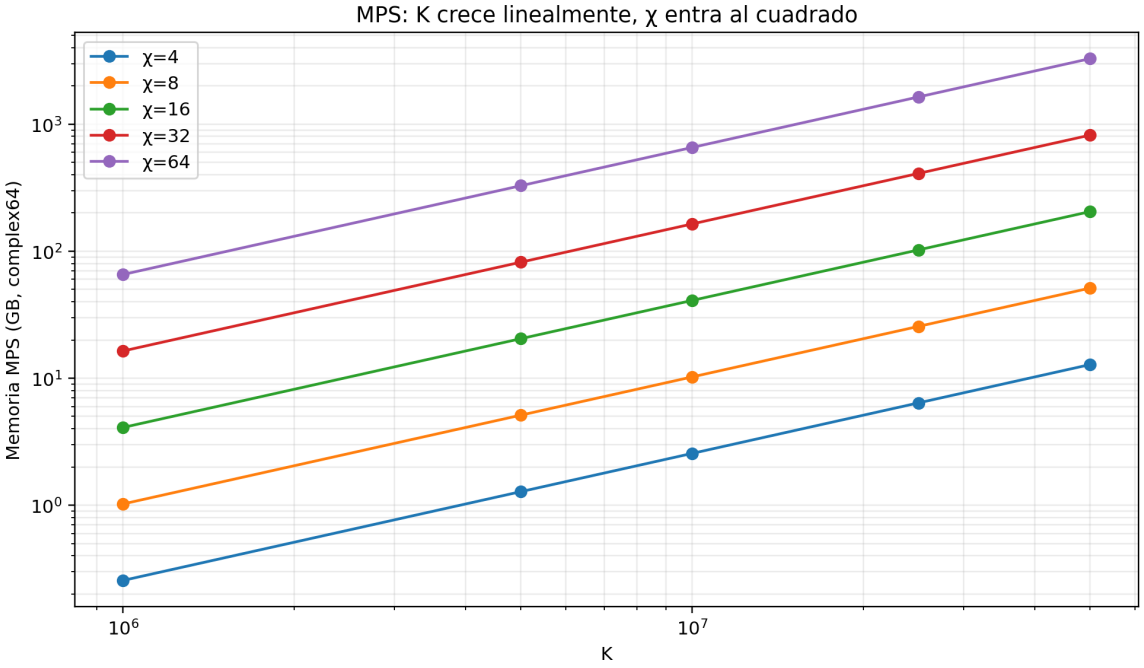


Figura 7: Sensibilidad de la memoria MPS a la dimensión de enlace.

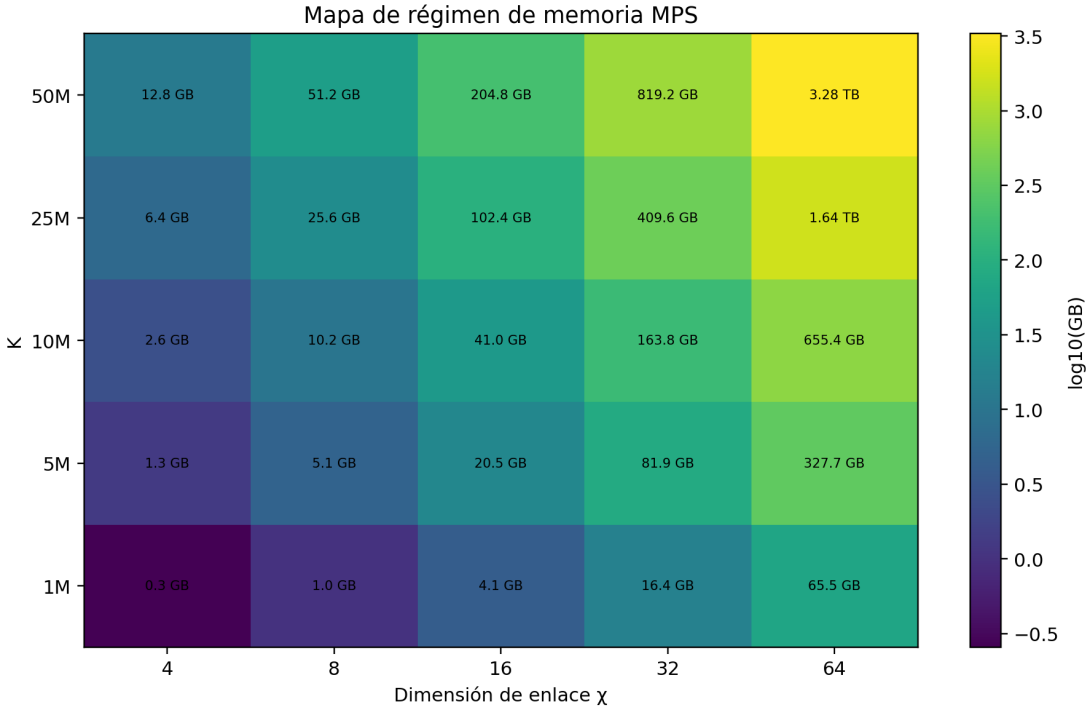


Figura 8: Mapa de regímenes de memoria MPS para diferentes K y χ.

8 El comportamiento de b: la geometría importa

En un QUBO bandeado o un grafo con vecindad local, el número de no nulos crece aproximadamente como

$$\text{nnz} \approx K(2b+1). \tag{7}$$

El ancho de banda b se convierte en la variable de estructura. Para $K = 50$ millones, nuestro modelo de almacenamiento CSR estima aproximadamente 4.4 GB para $b = 2$, 7.6 GB para $b = 4$, 14.0 GB

para $b = 8$, 26.8 GB para $b = 16$ y 52.4 GB para $b = 32$.

Esto conecta directamente con el uso de Reverse Cuthill–McKee propuesto en QAF: el reordenamiento no es una optimización cosmética. Si realmente logra convertir interacciones relevantes en una banda estrecha, cambia el régimen de memoria de cuadrático a aproximadamente lineal en K para b acotado [1].

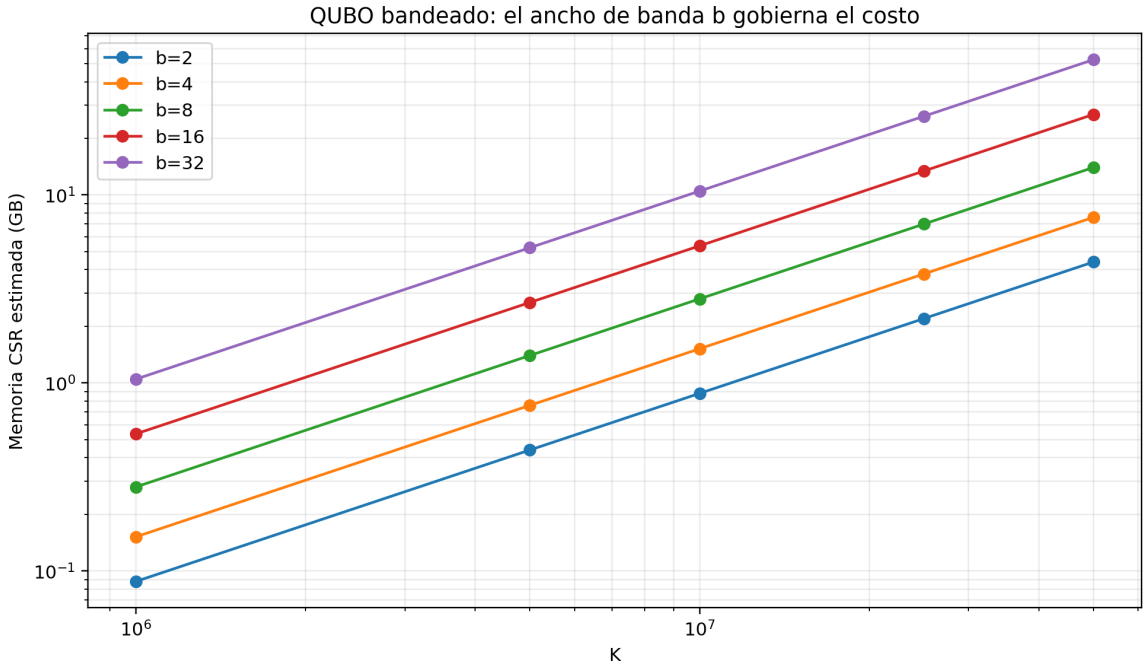


Figura 9: La memoria de una representación bandeada depende linealmente de K y del ancho b.

9 Resultados ejecutados: proxy streaming hasta 50 millones

El proxy fue ejecutado dos veces por tamaño, después de una etapa de calentamiento. La Tabla 2 resume los promedios.

Cuadro 2: Resultados medidos del proxy streaming.

K	Tiempo medio (s)	Throughput (Mvars/s)	Incremento máx. RSS observado (MB)
1 M	0.0213	46.89	39.87
5 M	0.1086	46.07	47.87
10 M	0.2255	44.58	47.87
25 M	0.5778	43.35	47.87
50 M	1.0689	46.78	47.87

El ajuste lineal entre tiempo y K produjo $R^2 = 0.998$. La pendiente corresponde a un throughput asintótico aproximado de 46.5 millones de variables por segundo para esta rutina y este entorno de ejecución. La estabilidad del incremento de memoria se explica porque el tamaño de bloque se mantuvo fijo: aumentar K aumenta el número de iteraciones de bloques, no el tamaño del bloque residente.

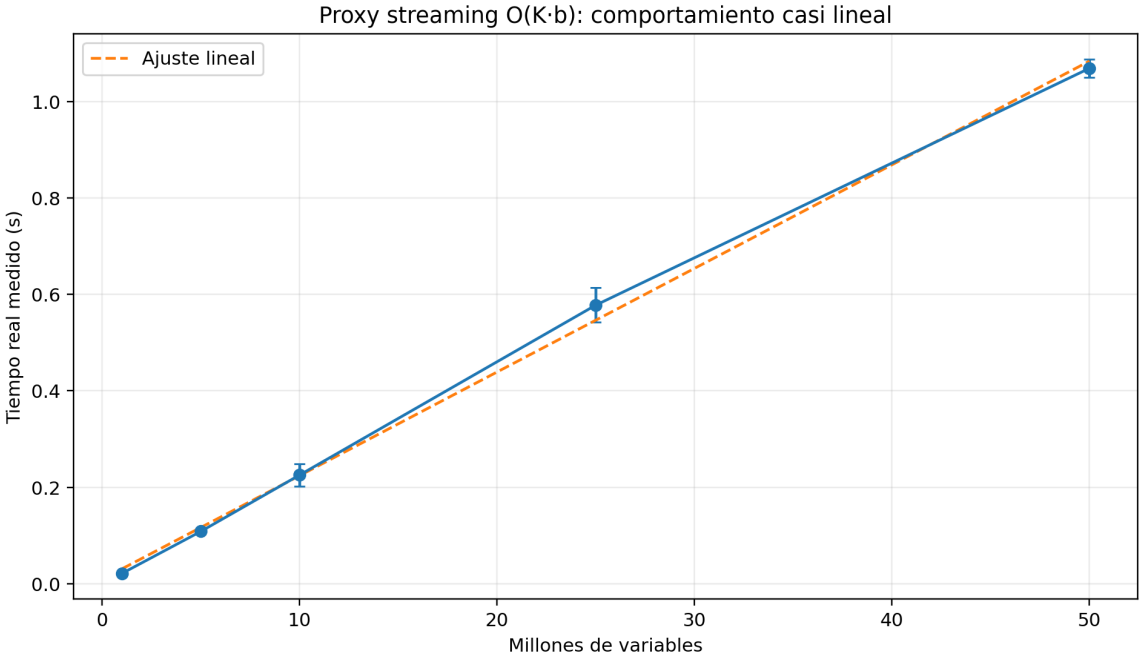


Figura 10: Tiempo medido del proxy streaming.

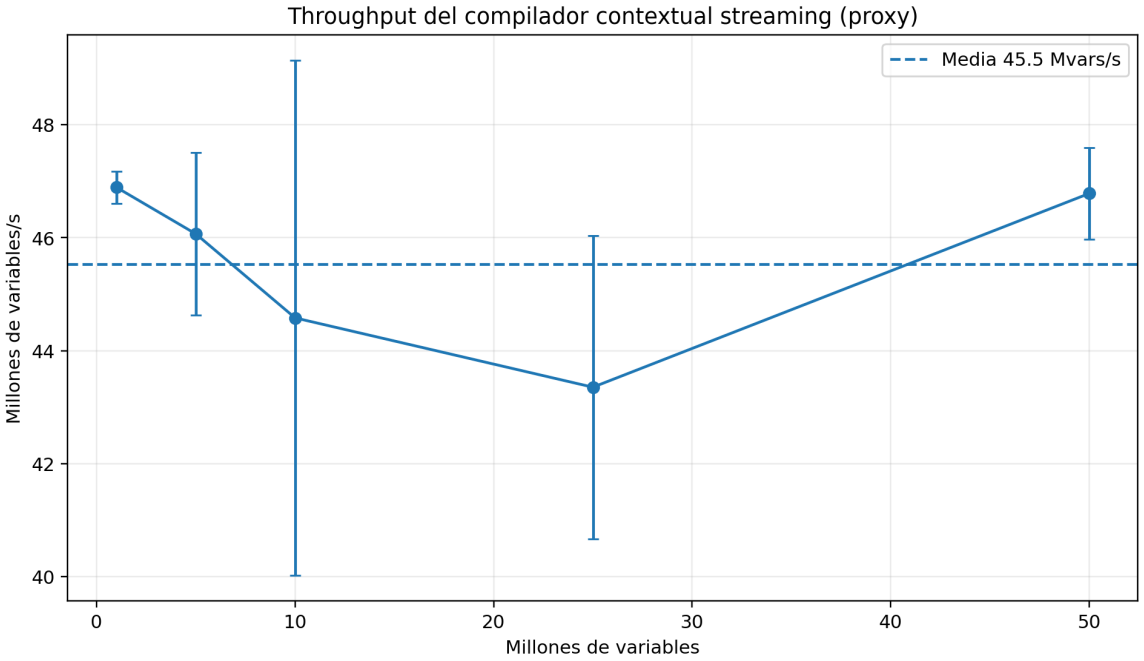


Figura 11: Throughput medido. La tasa permanece aproximadamente estable entre 1 y 50 millones.

10 Trabajo TDVP: lineal en K no significa barato

Una expresión $\mathcal{O}(K\chi^3)$ puede sonar "lineal" porque K aparece a la primera potencia. Sin embargo, el factor χ^3 es capaz de dominar el costo. Si χ pasa de 8 a 64, el término cúbico se multiplica por

$$\left(\frac{64}{8}\right)^3 = 512. \tag{8}$$

Por tanto, una arquitectura que escala a 50 millones con $\chi = 8$ puede dejar de ser práctica si la dinámica requiere $\chi = 64$. Este es un ejemplo clásico de cómo una fórmula asintótica aparentemente

favorable contiene un parámetro estructural capaz de cambiar completamente el régimen.

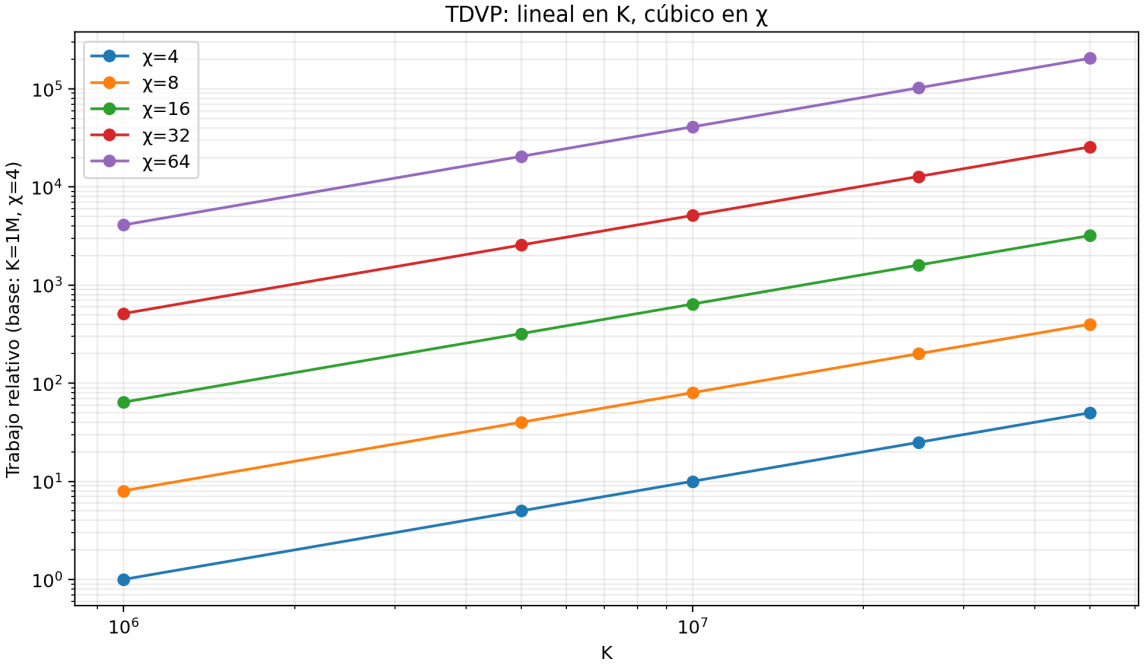


Figura 12: Trabajo relativo del término dominante $Kd^3\chi^3$.

11 Sabidurías halladas

11.1 1. El gran número relevante no es K, sino K^2 cuando todo interactúa con todo

Cincuenta millones de variables producen 1.25×10^{15} pares. Ese salto transforma un problema nominalmente grande en uno físicamente distinto. Una arquitectura escalable debe evitar representar o inspeccionar esos pares salvo cuando sean realmente necesarios.

11.2 2. La estructura vale más que la fuerza bruta

Una matriz de 20 PB no se vuelve conveniente con un solver ligeramente más rápido. Cambiar la representación de densa a bandeada o de densa a bajo rango altera órdenes de magnitud. El avance potencial de QAF, por tanto, debe demostrarse principalmente en *estructura* y no solo en velocidad del solver.

11.3 3. χ es una variable de fase computacional

Si χ se mantiene pequeño, MPS puede ser viable a enormes K. Si crece de forma agresiva, la ventaja se desvanece. Por eso un benchmark futuro debe medir $\chi(t)$, error de truncamiento, fidelidad/precisión y costo simultáneamente.

11.4 4. RCM y localidad deben convertirse en métricas, no en afirmaciones

La siguiente validación debería medir antes y después de RCM: ancho de banda, perfil, número de acoplamientos de largo alcance, memoria de la matriz resultante, calidad de decisión y tiempo de compilación. Solo así se sabrá si la topología realmente preserva la información decisional mientras reduce complejidad.

11.5 5. 50 millones exigen arquitectura jerárquica

El límite actual documentado del BQM híbrido de D-Wave es un millón de variables [7]. Incluso cuando la representación local sea viable, 5–50 millones requieren particionar y coordinar subproblemas. Una ruta razonable es

$$K \gg 10^6 \rightarrow \text{clusters locales} \rightarrow \text{QUBOs parciales} \rightarrow \text{coordinación global.} \tag{9}$$

Esto sugiere una extensión natural del QAF: un QAF jerárquico o multiescala.

11.6 6. La prueba streaming muestra una posibilidad, no una solución completa

Procesar 50 millones de variables en cerca de un segundo fue posible porque la conectividad estaba acotada y la memoria fue tratada por bloques. Esa es una demostración útil de comportamiento lineal en una etapa local. No demuestra que TDVP, RDM, optimización y readout completos tengan ese rendimiento. El valor científico de la prueba está en separar qué parte ya escala y qué parte permanece como pregunta abierta.

12 Un programa experimental para validar $K = 50$ millones de forma rigurosa

Proponemos cinco puertas experimentales:

Fase 1. Representación: construir grafos sintéticos y reales con K creciente; medir banda antes/después de RCM y memoria sparse real.

Fase 2. MPS: ejecutar MPS reales en escalas crecientes y registrar χ requerido para mantener un error objetivo. El criterio de éxito debe ser explícito, por ejemplo error $< \epsilon$ con $\chi \leq \chi_{max}$.

Fase 3. TDVP: medir tiempo por sweep y error acumulado, no solo la complejidad teórica.

Fase 4. Compilación QUBO: generar QUBOs por bloques/ventanas sin materializar K^2; medir throughput, ancho, número de no nulos y calidad del objetivo.

Fase 5. Solución jerárquica: para $K > 10^6$, particionar en subproblemas, resolver localmente y evaluar el costo de coordinación global frente a un baseline clásico distribuido.

El criterio de fracaso también debe quedar definido: si χ crece rápidamente con K o si la partición reduce de manera inaceptable la calidad de decisión, la hipótesis de escalabilidad debe revisarse.

13 Implicaciones para una propuesta de investigación avanzada

Este estudio cambia la narrativa de una afirmación amplia a una pregunta técnica precisa. No conviene decir: "QAF escala a 50 millones de variables". La formulación defendible es:

> **QAF propone mecanismos estructurales que podrían hacer viable el procesamiento de sistemas con decenas de millones de variables si la conectividad efectiva puede mantenerse local/bandeada y si la dimensión de enlace necesaria permanece acotada. Hemos validado analíticamente los regímenes de memoria y ejecutado una etapa local streaming hasta 50 millones; la siguiente incertidumbre técnica es demostrar que la información decisional real puede conservarse bajo esas restricciones.**

Esta frase contiene una ventaja: puede ser falsada. Y precisamente por ello constituye una mejor proposición de investigación.

14 Limitaciones

- Los resultados a $K = 1$–50 millones del proxy son resultados de una pasada contextual local, no de un QAF MPS/TDVP completo.
- Las estimaciones MPS suponen almacenamiento complex64 y usan una cota interior simplificada $Kd\chi^2$; implementaciones reales añaden overhead, bordes, operadores y buffers.
- La estimación CSR usa 8 bytes para valor, 8 para índice y 8 para puntero de fila, como modelo conservador; bibliotecas concretas pueden usar tamaños distintos.
- No se ejecutó un solver de optimización sobre $K = 50$ millones.
- No se evaluó aún cómo crece χ con incertidumbre, densidad del grafo o no conmutatividad operacional real.
- Los tiempos medidos dependen del hardware y del entorno actual; deben reproducirse en infraestructura controlada antes de utilizarse como benchmark externo.

15 Conclusiones

El estudio muestra que los grandes números adquieren significado únicamente cuando se especifica la estructura. Un QUBO denso de 50 millones de variables exige alrededor de 20 PB y es impráctico como objeto explícito. El mismo K, bajo una banda $b = 8$, se reduce a un orden de 14 GB en CSR. Una representación MPS puede encontrarse entre decenas de gigabytes y varios terabytes según χ. El proxy streaming mostró tiempo casi lineal hasta 50 millones y memoria incremental aproximadamente constante por uso de bloques.

La conclusión principal es, por tanto, una regla de diseño:

$$\boxed{\text{Escalabilidad QAF} \neq f(K) \quad \text{solamente;} \qquad \text{Escalabilidad QAF} = f(K, b, \chi, \rho, \epsilon).} \tag{10}$$

Aquí b representa localidad/banda, χ la complejidad de correlación, ρ la densidad efectiva de interacciones y ϵ el error tolerado. La siguiente fase debe caracterizar experimentalmente esas funciones. Si QAF puede mantener b y χ pequeños sin perder calidad decisional, la escala de decenas de millones deja de ser una fantasía de fuerza bruta y se convierte en un problema de estructura computacional. Si no puede hacerlo, el límite aparecerá de manera medible. En ambos casos, el experimento produce conocimiento útil.

Referencias

Referencias

[1] A. Bancells, *Quantum Analysis Framework (QAF): dinámica operacional no conmutativa y compilación QUBO asíncrona para la toma de decisiones complejas bajo incertidumbre*, propuesta técnica ejecutiva, 2026.

[2] C. D. Rojas Rodríguez, *Arquitectura quantum-like multiagente para apoyo a decisiones críticas en entornos de defensa bajo incertidumbre*, UNIR / ParkiApp Research, 2026.

[3] I. V. Oseledets, "Tensor-Train Decomposition," *SIAM Journal on Scientific Computing*, 33(5), 2295–2317, 2011. DOI: 10.1137/090752286.

[4] U. Schollwöck, "The density-matrix renormalization group in the age of matrix product states," *Annals of Physics*, 326(1), 96–192, 2011.

[5] J. Haegeman et al., "Unifying time evolution and optimization with matrix product states," *Physical Review B*, 94, 165116, 2016.

[6] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, 2, 5, 2014.

[7] D-Wave Quantum, *BQM Solver Properties*, documentación oficial consultada el 19-09-2026. La documentación reporta `maximum_number_of_variables = 1000000` para el solver híbrido BQM. https://docs.dwavequantum.com/en/latest/industrial_optimization/solver_bqm_properties.html

[8] Gurobi Optimization, *Python Low Level Model Building Reference* y *Model Attributes*, documentación oficial consultada el 19-09-2026. https://docs.gurobi.com/

A Código Python completo de la prueba de escalabilidad

```python
#!/usr/bin/env python3
"""
QAF scalability study: large-K representation and streaming compilation proxy.

Scientific scope
----------------
This program does NOT instantiate or solve a 1M-50M variable full QAF MPS/TDVP
model. It does two complementary things:

1) Computes exact storage/interaction scaling laws for dense QUBO, sparse banded
   QUBO, and fixed-bond-dimension MPS representations.
2) Executes a chunked O(K*b) streaming contextual-compilation proxy at
   K = 1M, 5M, 10M, 25M, 50M to measure practical linear-pass throughput
   without materializing a KxK matrix.

The streaming proxy is a scalability harness, not evidence that the complete QAF
pipeline runs at these K values. It isolates a necessary property for such scale:
linear-memory, locality-bounded processing.
"""

from __future__ import annotations
import argparse
import math
import os
import time
from pathlib import Path
import numpy as np
import pandas as pd
import psutil

K_VALUES = np.array([1_000_000, 5_000_000, 10_000_000, 25_000_000, 50_000_000], dtype=np.int64)
CHI_VALUES = [4, 8, 16, 32, 64]
BANDWIDTHS = [2, 4, 8, 16, 32]

DECIMAL = 1000.0

def human_bytes(x: float) -> str:
    if x < 1e3: return f"{x:.0f} B"
    units = ["KB","MB","GB","TB","PB","EB"]
    v = float(x)
    for u in units:
        v /= 1000.0
        if v < 1000.0:
            return f"{v:.3g} {u}"
    return f"{v:.3e} B"


def dense_qubo_bytes(K: int, symmetric_packed: bool=False) -> float:
    elems = K * (K + 1) / 2 if symmetric_packed else K * K
    return elems * 8.0 # Float64


def sparse_banded_nnz(K: int, b: int) -> int:
    # Symmetric full band with diagonal; edge correction exact for K > b.
    if K <= b:
        return K * K
    return int(K * (2*b + 1) - b * (b + 1))


def csr_bytes(K: int, b: int) -> float:
    # Conservative 64-bit data + 64-bit column index + 64-bit row pointer.
    nnz = sparse_banded_nnz(K, b)
    return nnz * 16.0 + (K + 1) * 8.0


def mps_bytes(K: int, chi: int, d: int=2, bytes_per_complex: int=8) -> float:
    # complex64 = 8 bytes. Interior-site upper-bound parameter count K*d*chi^2.
    return float(K) * d * chi * chi * bytes_per_complex


```

```python
def tdvp_work_units(K: int, chi: int, d: int=2) -> float:
    # Normalized dominant scaling proxy O(K*d^3*chi^3); not seconds.
    return float(K) * (d**3) * (chi**3)


def build_theory_table() -> pd.DataFrame:
    rows = []
    for K in K_VALUES:
        row = {
            "K": int(K),
            "dense_qubo_bytes": dense_qubo_bytes(int(K)),
            "dense_qubo_packed_bytes": dense_qubo_bytes(int(K), True),
            "pairs": int(K) * (int(K) - 1) // 2,
        }
        for b in BANDWIDTHS:
            row[f"csr_b{b}_bytes"] = csr_bytes(int(K), b)
            row[f"csr_b{b}_nnz"] = sparse_banded_nnz(int(K), b)
        for chi in CHI_VALUES:
            row[f"mps_chi{chi}_bytes"] = mps_bytes(int(K), chi)
            row[f"tdvp_chi{chi}_work"] = tdvp_work_units(int(K), chi)
        rows.append(row)
    return pd.DataFrame(rows)


def hash_uniform(u: np.ndarray, salt: np.uint64) -> np.ndarray:
    """Fast deterministic [0,1) values from uint64 ids using SplitMix-like mixing."""
    x = u + salt
    x = (x ^ (x >> np.uint64(30))) * np.uint64(0xbf58476d1ce4e5b9)
    x = (x ^ (x >> np.uint64(27))) * np.uint64(0x94d049bb133111eb)
    x = x ^ (x >> np.uint64(31))
    # 53 high-quality bits -> float64 [0,1)
    return ((x >> np.uint64(11)).astype(np.float64)) * (1.0 / 9007199254740992.0)


def streaming_proxy(K: int, block: int=500_000, offsets=(1,2,4,8), gamma: float=0.15) -> dict:
    """Chunked O(K*b) contextual compilation proxy.

    Per variable constructs deterministic confidence, damage, local context and a
    scalar QUBO-like diagonal score, then accumulates local pair interactions for
    fixed offsets. No KxK matrix is materialized.
    """
    proc = psutil.Process(os.getpid())
    rss0 = proc.memory_info().rss
    peak_rss = rss0
    checksum_diag = 0.0
    checksum_pair = 0.0
    positive = 0
    n_done = 0

    # Keep an overlap tail so pair terms across chunk boundaries are counted.
    max_off = max(offsets)
    prev_tail = np.empty(0, dtype=np.float64)

    t0 = time.perf_counter()
    for start in range(0, K, block):
        end = min(K, start + block)
        u = np.arange(start, end, dtype=np.uint64)
        p = 0.05 + 0.90 * hash_uniform(u, np.uint64(0x123456789abcdef0))
        dmg = 0.30 + 0.70 * hash_uniform(u, np.uint64(0x9e3779b97f4a7c15))
        ctx = hash_uniform(u, np.uint64(0x243f6a8885a308d3)) - 0.5
        # Contextual update bounded to [0.005, 0.995].
        p_ctx = np.clip(p + gamma * ctx * p * (1.0 - p), 0.005, 0.995)
        score = p_ctx * dmg - 0.05

        checksum_diag += float(score.sum(dtype=np.float64))
        positive += int(np.count_nonzero(score > 0.0))

        if prev_tail.size:
            joined = np.concatenate([prev_tail, score])
            base_start = max_off
        else:
            joined = score
            base_start = 0
```

```
        # Count interactions whose left endpoint belongs to current output region,
        # with bounded local offsets. This is an arithmetic proxy for banded terms.
        for off in offsets:
            if joined.size > off:
                left = joined[:-off]
                right = joined[off:]
                if prev_tail.size:
                    # use the portion that includes current chunk and cross-boundary terms
                    sidx = max(0, prev_tail.size - off)
                    checksum_pair += float(np.dot(left[sidx:], right[sidx:]))
                else:
                    checksum_pair += float(np.dot(left, right))

        prev_tail = score[-max_off:].copy() if score.size >= max_off else score.copy()
        n_done += (end - start)
        peak_rss = max(peak_rss, proc.memory_info().rss)

    elapsed = time.perf_counter() - t0
    return {
        "K": int(K),
        "elapsed_s": elapsed,
        "throughput_Mvars_s": (K / elapsed) / 1e6,
        "peak_delta_MB": max(0, peak_rss - rss0) / 1e6,
        "rss_end_MB": proc.memory_info().rss / 1e6,
        "checksum_diag": checksum_diag,
        "checksum_pair": checksum_pair,
        "positive_fraction": positive / K,
        "offset_count": len(offsets),
        "block": block,
    }


def benchmark_streaming(repeats: int=2, block: int=500_000) -> pd.DataFrame:
    rows = []
    # warm-up
    streaming_proxy(100_000, block=min(block, 100_000))
    for K in K_VALUES:
        for rep in range(repeats):
            r = streaming_proxy(int(K), block=block)
            r["repeat"] = rep + 1
            rows.append(r)
            print(f"K={K:,} rep={rep+1}: {r['elapsed_s']:.3f}s, {r['throughput_Mvars_s']:.2f} Mvars/s")
    return pd.DataFrame(rows)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", default="results")
    ap.add_argument("--repeats", type=int, default=2)
    ap.add_argument("--block", type=int, default=500_000)
    args = ap.parse_args()
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)

    theory = build_theory_table()
    theory.to_csv(out / "scaling_theory.csv", index=False)

    stream = benchmark_streaming(args.repeats, args.block)
    stream.to_csv(out / "streaming_benchmark_raw.csv", index=False)
    summary = stream.groupby("K").agg(
        elapsed_s_mean=("elapsed_s","mean"),
        elapsed_s_std=("elapsed_s","std"),
        throughput_Mvars_s_mean=("throughput_Mvars_s","mean"),
        throughput_Mvars_s_std=("throughput_Mvars_s","std"),
        peak_delta_MB_max=("peak_delta_MB","max"),
        positive_fraction=("positive_fraction","mean"),
    ).reset_index()
    summary.to_csv(out / "streaming_benchmark_summary.csv", index=False)

    print("\nTheory summary")
    for _, r in theory.iterrows():
        K=int(r.K)
        print(f"K={K:,}: dense QUBO={human_bytes(r.dense_qubo_bytes)}, "
```

```
            f"banded b=8={human_bytes(r.csr_b8_bytes)}, MPS chi=16={human_bytes(r.mps_chi16_bytes)}")

if __name__ == "__main__":
    main()
```